UNIT-3
Nested Queries, SubQueries

Query(Select):
A query is a command used to get data from a database table.
Syntax:
select column_name1, column_name2, ...
from table_name
where condition;

A **Subquery (Nested Query)** is a query written inside another query. The inner query is called the subquery, and the outer query is called the main query. The subquery executes first and returns a result, which may be a single value or multiple values. The main query uses that result and then executes to produce the final output.
Syntax:
SELECT column_name1, column_name2, ...
FROM table_name
WHERE column_name **operator** (
  SELECT column_name
  FROM table_name
  WHERE condition
);
The inner query must return:

- **Single value** → use =, >, <
- **Multiple values** → use IN, ANY, ALL, EXISTS, and NOT EXISTS.

Various **Operators** are used in nested queries like
IN
NOT IN
ANY
ALL
EXIST
NOT EXIST
LIKE
BETWEEN
Consider the following Tables

EMP TABLE

empno	**ename**	**sal**	**deptno**
	Ram	3000	10
102	Sita	4000	20
103	Ravi	2500	10
104	Gita	5000	30
105	Ramesh	3500	20

DEPT TABLE

deptno	dname
10	HR
20	SALES
30	IT

1. Subquery using IN

The IN operator is used to check if a value exists in a given list or in the result of a subquery.

Example: Find employees working in SALES department.

```
SELECT *
FROM emp
WHERE deptno IN (
   SELECT deptno
   FROM dept
   WHERE dname = 'SALES'
);
```

Output:

empno	ename	Sal	deptno
102	Sita	4000	20
105	Ramesh	3500	20

2.SubQuery using NOT IN

The NOT IN operator is used to check if a value is not included in a list or in the result of a subquery.

Example: Find employees who are not working in the HR department.

```
SELECT *
FROM emp
WHERE deptno NOT IN (
   SELECT deptno
   FROM dept
   WHERE dname = 'HR'
);
```

empno	ename	sal	deptno
102	Sita	4000	20
104	Gita	5000	30
105	Ramesh	3500	20

3.SubQuery using ANY

The ANY operator is used in SQL to compare a value with any one of the values returned by a subquery.It must be used with comparison operators such as =, >, <, >=, <=, or <>. The condition is satisfied if at least one value in the subquery result meets the comparison condition.

Example: Find employees earning more than ANY employee in dept 10

```
SELECT *
FROM emp
WHERE sal > ANY (
   SELECT sal
   FROM emp
   WHERE deptno = 10
);
```

Output:

empno	ename	sal	deptno
101	Ram	3000	10
102	Sita	4000	20
104	Gita	5000	30
105	Ramesh	3500	20

3. SubQuery using ALL

The ANY operator is used to compare a value with any one of the values returned by a subquery. The condition is true if at least one value in the subquery satisfies the comparison.

Example: Find employees earning more than ALL employees in dept 10

```
SELECT *
FROM emp
WHERE sal > ALL (
   SELECT sal
   FROM emp
   WHERE deptno = 10
);
```

empno	ename	sal	deptno
102	Sita	4000	20
104	Gita	5000	30
105	Ramesh	3500	20

4. LIKE Operator

Used for **pattern matching**.

Symbols:

- % → Any number of characters
- _ → Exactly one character

Example: Find employee name whose name starts with 'R'

```
SELECT ename
FROM emp
WHERE ename LIKE 'R%';
```

Output:
Ram, Ravi, Ramesh

Example: Find employee names whose name ends with 'a'

```
SELECT ename
FROM emp
WHERE ename LIKE '%a';
```

Output:
Sita, Gita

Example: Find names with 4 letters

```
SELECT ename
FROM emp
WHERE ename LIKE '____';
```

Output:

Sita,Ravi, Gita

Example: Find the names whose 3rd letter is 'm'

```
SELECT ename
FROM emp
WHERE ename LIKE '__m%';
```
Output:

ram, ramesh

Example: Find employee name whose name does not start with 'S' and contains 'A'

```
SELECT ename
FROM emp
WHERE ename NOT LIKE 'S%'
AND ename LIKE '%A%';
```
Output:
Ravi
Gita
Ramesh

5. BETWEEN Operator

Find employees whose salary is between 3000 and 4000

```
SELECT *
FROM emp
WHERE sal BETWEEN 3000 AND 4000;
```

6. Subquery using EXISTS (Correlated Query)

Correlated Query

A correlated query is a subquery that depends on the outer query for its execution.
In correlated queries, the inner query is executed repeatedly, once for each row processed by the outer.

The EXISTS operator is used in correlated queries in SQL.

Example: Find departments that have employees

```
SELECT *
FROM dept d
WHERE EXISTS (
   SELECT *
   FROM emp e
   WHERE e.deptno = d.deptno
);
```

deptno	dname
10	HR
20	SALES
30	IT

7. Subquery using NOT EXISTS

Example: Find departments that have NO employees

```
SELECT *
FROM dept d
WHERE NOT EXISTS (
   SELECT *
   FROM emp e
   WHERE e.deptno = d.deptno
);
```

8. Nested Query

Example: Find employee having highest salary

```
SELECT *
FROM emp
WHERE sal = (
   SELECT MAX(sal)
   FROM emp
);
```

Output:

Empno	ename	sal	deptno
104	Gita	5000	30

Example: Find employee having third highest salary

```
SELECT *
FROM emp
WHERE sal = (
   SELECT MAX(sal)
   FROM emp
   WHERE sal < (
      SELECT MAX(sal)
      FROM emp
      WHERE sal < (
         SELECT MAX(sal)
         FROM emp
      )
   )
);
```

Output:

empno	ename	sal	deptno
105	Ramesh	3500	20

Example: Find employee with highest salary using RANK()

```
SELECT *
FROM (
   SELECT empno, ename, sal,
         RANK() OVER (ORDER BY sal DESC) AS rnk
   FROM emp
) t
WHERE rnk = 1;
```

GROUP BY Clause

The **GROUP BY** clause is used to **group rows that have the same value** in one or more columns.
Group by clause often used with aggregate functions.
The main aggregate functions are:
COUNT()– used to count the number of rows
MAX() – used to find the highest value in a column
MIN()– used to find the lowest value in a column
SUM() – used to find the total of values in a column
AVG() – used to find the average of values in a column

Rule: Which column name is used after group by clause that column name should be written after select keyword.

Syntax:

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Having Clause:

Having clause is used with group by only. In grouping we cann't use "where" clause so instead of "where" we will use Having Clause.
Or
To select the group on the basis of specified condition

Syntax:
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;

Order By clause:
Used to sort the result of SQL Query over the specified attribute.
SELECT column_name
FROM table_name
ORDER BY column_name [ASC | DESC];

Aggregate Functions:

COUNT():
Example: Count the records in the employee table.
SELECT COUNT(*) FROM emp;
Example: Count the number of distinct departments in the employee table.
SELECT COUNT(DISTINCT deptno) FROM emp**;**
MAX():
Example: Find the maximum/highest salary in the employee table.
SELECT MAX(sal) FROM emp;
MIN():
Example: Find the minimum/lowest salary in the employee table.
SELECT MIN(sal) FROM emp;
AVG():
Example: Find the average salary of employees.
SELECT AVG(sal) FROM emp;
SUM():
Example: Find the total/sum of salaries of employees.
SELECT SUM(sal) FROM emp;

GROUP BY (Using EMP Table)
Example: Count number of employees in each department
SELECT deptno, COUNT(empno)
FROM emp
GROUP BY deptno;
Output:

Deptno	count
10	2
20	2
30	1

HAVING Clause:
1. **HAVING** is used to filter **grouped results**.
2. Used after GROUP BY.
Example: Find departments having more than 1 employee
SELECT deptno, COUNT(empno)
FROM emp

GROUP BY deptno
HAVING COUNT(empno) > 1;

Output:

Deptno	count
10	2
20	2

Ordering/Order By Clause:

It is used to convert the relation into ascending or descending order by default ascending order retruns. If we want to convert descending order then write “desc” after order by clause

Example: select ename from emp order by name desc;

Output:

Ename
Sita
Ramesh
Ravi
Ram
Gita